

Verilog 大作业实验报告

学生姓名	杨滢	学号	2412266	指导老师	时晶磊
------	----	----	---------	------	-----

1 案例二：两个操作数的算数运算

1.1 电路功能说明

实现两个操作数的算数运算（加减乘除）、移位运算（左移右移）

1.2 电路设计

1.2.1 输入输出

输入：a、b 两个 32 位操作数，enc1、enc2 两个操作数的编码类型，定义对应关系 00 原码，01 反码，10 补码，shift 移位位数；输出：6 种运算后得到的计算结果，**结果统一以补码形式表示。**

1.2.2 编码转换模块

将不同编码格式转换为统一的补码表示。00 原码：负数取反加 1，正数保持不变；01 反码：负数加 1 转换为补码；10 补码：直接输出。

1.2.3 运算模块

同时进行所有算术和移位运算，所有运算结果并行输出。

1.3 电路实现

```
1 //顶层实现
2 `timescale 1ns / 1ps
3 module finalproject2(
4     input [31:0] a, b,
5     input [1:0] enc1, enc2,
6     input [4:0] shift,
7     output [31:0] add, sub, mul, div, shl,
8         shr );
9 wire [31:0] a_comp, b_comp;
10 encoder_converter encoder_a(
11     .data_in(a),
12     .enc_type(enc1),
13     .data_out(a_comp));
14 encoder_converter encoder_b(
15     .data_in(b),
16     .enc_type(enc2),
17     .data_out(b_comp));
18 arithmetic_unit arithmetic(
19     .a_comp(a_comp),
20     .b_comp(b_comp),
21     .add(add),
22     .sub(sub),
23     .mul(mul),
24     .div(div));
25 shift_unit shifter(
26     .data_in(a_comp),
27     .shift_amount(shift),
28     .shl(shl),
29     .shr(shr));
```

```

29 endmodule
30 //编码转换模块
31 `timescale 1ns / 1ps
32 module encoder_converter(
33     input [31:0] data_in,
34     input [1:0] enc_type,
35     output reg [31:0] data_out );
36 always @(*) begin
37     case(enc_type)
38         2'b00: begin
39             if (data_in[31])
40                 data_out = (~{1'b0,
41                     data_in[30:0]} + 1) |
42                     {1'b1, 31'b0};
43             else
44                 data_out = data_in;
45             end
46         2'b01: begin
47             if (data_in[31])
48                 data_out = data_in + 1;
49             else
50                 data_out = data_in;
51             end
52         2'b10: begin
53             data_out = data_in;
54             end
55         default: data_out = data_in;
56     endcase
57 end
58 endmodule

```

```

1 //加减乘除计算模块核心代码

```

```

2 always @(*) begin
3     add = a_comp + b_comp;
4     sub = a_comp - b_comp;
5     mul = $signed(a_comp) *
6         $signed(b_comp);
7     if (b_comp == 0)
8         div = 0;
9     else
10        div = $signed(a_comp) /
11            $signed(b_comp);
12 end

```

```

11 //加减乘除计算模块核心代码

```

```

12 always @(*) begin
13     add = a_comp + b_comp;
14     sub = a_comp - b_comp;
15     mul = $signed(a_comp) *
16         $signed(b_comp);
17     if (b_comp == 0)
18         div = 0;
19     else
20        div = $signed(a_comp) /
21            $signed(b_comp);
22 end

```

```

20 end
21 //位移计算模块核心代码

```

```

22 always @(*) begin
23     shl = data_in << shift_amount;
24     shr = $signed(data_in) >>>
25         shift_amount;
26 end

```

1.4 功能验证及电路图

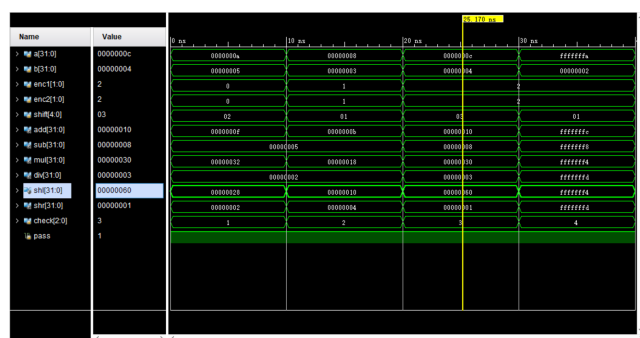


图 1: 波形图

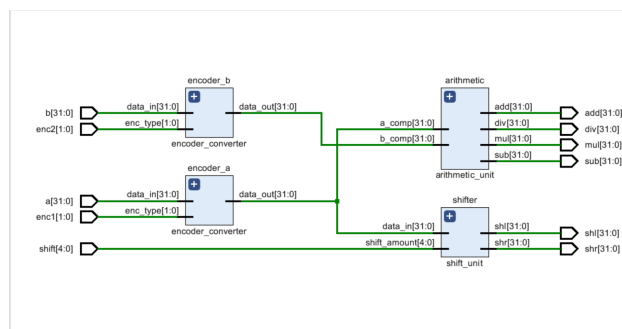


图 2: 电路图

编码类型设置：00 原码 01 反码 10 补码。

4 组测试案例，一组正数原码 (10 和 5)、一组正数反码 (8 和 3)、一组正数补码 (12 和 4)、一组负数补码 (-6 和 2)。波形图中为直观表现结果正确性，增加一个 pass，如果输出结果和预期结果相同 pass 为 1，否则为 0。

2 案例三：投票评分电路

2.1 电路功能说明

9 名裁判打分，分数 0-100，当 9 名裁判全部给出分数时，去掉一个最高分和一个最低分，计算最终平均得分。

2.2 电路设计

2.2.1 输入输出

输入：时钟信号 clk、9 位有效标志 valid、9 个 7 位数据 s0-s8；输出：7 位平均值 avg

2.2.2 最小值/最大值查找模块

通过四级比较树找到 9 个数中的最小值和最大值。

2.2.3 平均值计算

只有输入 9 个有效分数，即 9 个 valid 信号全为 1 时，系统才会给平均值 avg 赋除去最大最小值的分数总和除以 7 后的值，否则 avg 输出为 0。

2.3 电路实现

```

1 // 顶层模块
2 module finalproject3(
3     input clk,
4     input [8:0] valid,
5     input [6:0] s0,s1,s2,s3,s4,s5,s6,s7,s8,
6     output reg [6:0] avg );
7 wire [6:0] min_val, max_val;
8 wire [9:0] total_sum;
9 find_min u_find_min(
10     .s0(s0), .s1(s1), .s2(s2), .s3(s3),
11     .s4(s4),
12     .s5(s5), .s6(s6), .s7(s7), .s8(s8),
13     .min_val(min_val));
14 find_max u_find_max(
15     .s0(s0), .s1(s1), .s2(s2), .s3(s3),
16     .s4(s4),
17     .s5(s5), .s6(s6), .s7(s7), .s8(s8),
18     .max_val(max_val));
19
20 sum_calculator u_sum_calculator(
21     .s0(s0), .s1(s1), .s2(s2), .s3(s3),
22     .s4(s4),
23     .s5(s5), .s6(s6), .s7(s7), .s8(s8),
24     .min_val(min_val),
25     .max_val(max_val),
26     .total_sum(total_sum));
27
28 initial begin
29     avg = 0;
30 end
31
32 \\先对输入合法性进行判断
33 always @(posedge clk) begin
34     if (valid != 9'b111111111) begin
35         avg <= 0;
36     end else begin
37         avg <= total_sum / 7;
38     end
39 end

```

```

33 endmodule
34
35 //最小值查找模块核心代码
36 reg [6:0] temp_min;
37 always @(*) begin
38     //四级比较
39     temp_min = (s0 < s1) ? ((s0 < s2) ? s0
40         : s2) : ((s1 < s2) ? s1 : s2);
41     temp_min = (temp_min < s3) ?
42         ((temp_min < s4) ? temp_min : s4) :
43         ((s3 < s4) ? s3 : s4);
44     temp_min = (temp_min < s5) ?
45         ((temp_min < s6) ? temp_min : s6) :
46         ((s5 < s6) ? s5 : s6);
47     temp_min = (temp_min < s7) ?
48         ((temp_min < s8) ? temp_min : s8) :
49         ((s7 < s8) ? s7 : s8);
50     min_val = temp_min;
51 end
52
53 //最大值查找模块核心代码
54 reg [6:0] temp_max;
55 always @(*) begin
56     temp_max = (s0 > s1) ? ((s0 > s2) ? s0
57         : s2) : ((s1 > s2) ? s1 : s2);
58     temp_max = (temp_max > s3) ?
59         ((temp_max > s4) ? temp_max : s4) :
60         ((s3 > s4) ? s3 : s4);
61     temp_max = (temp_max > s5) ?
62         ((temp_max > s6) ? temp_max : s6) :
63         ((s5 > s6) ? s5 : s6);
64     temp_max = (temp_max > s7) ?
65         ((temp_max > s8) ? temp_max : s8) :
66         ((s7 > s8) ? s7 : s8);
67     max_val = temp_max;
68 end
69
70 //求和模块核心代码
71 total_sum = s0 + s1 + s2 + s3 + s4 + s5 +
72     s6 + s7 + s8 - min_val - max_val;

```

2.4 功能验证及电路图

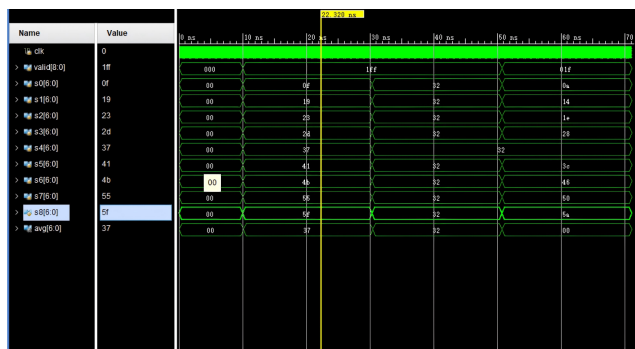


图 3: 波形图

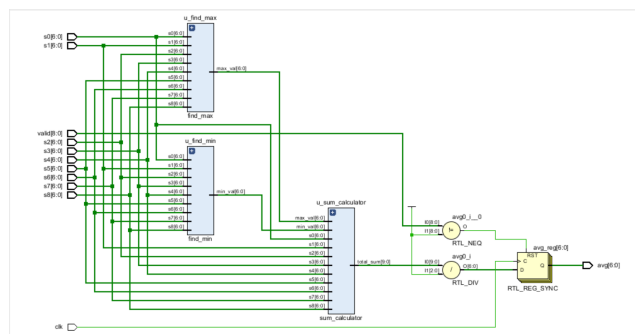


图 4: 电路图

3 组测试案例，第一组随机范围内的 9 个分数，第二组 9 个相同的分数，第三组 5 个随机分数。

3 实验总结

1. 通过这次大作业，我学会了分模块设计电路，这能提高代码的可重复使用性。
2. 掌握了时钟同步设计和数据有效性验证机制。
3. 提高了解决具体问题的能力。从一个具有现实意义功能的题干开始，逐步构思解决。
4. 对 vivado 绘制电路图功能有了一定了解，对该软件更加熟悉。